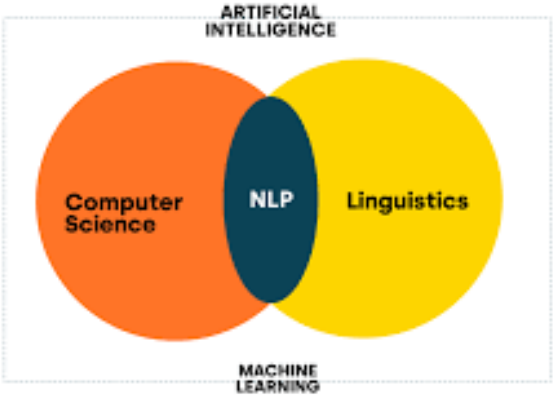
Natural Language Processing (NLP)
24BAM6C10

UNIT I

1. Introduction: Computing with Language

Natural Language Processing (NLP) is an AI field combining computer science, linguistics, and machine learning to help computers understand, interpret, and generate human language (text and speech). It bridges the gap between human communication and machine



It focuses on enabling computers to process, understand, and generate human language. The phrase *"Computing with Language"* refers to the computational treatment of linguistic data—how computers can be programmed to handle text and speech in natural languages.

In the modern era, language data is abundant in the form of emails, web pages, news articles, blogs, and social media posts. The challenge is to convert this raw, unstructured text into structured data that computers can understand and use for various applications such as

- Search engines
- Chatbots
- Translation systems
- Sentiment analysis
- Spam & Fraud Detection
- Information Extraction
- Speech-to-Text and Text-to-Speech

Key Objectives of Language Processing:

1. Understanding linguistic structure (syntax, semantics, pragmatics)
2. Extracting information and meaning from text
3. Generating natural language text

4. Building systems that can understand or produce language automatically

Python has emerged as a preferred programming language for NLP due to its simplicity and the availability of powerful libraries like **NLTK (Natural Language Toolkit)**, **spaCy**, and **transformers**.

NLTK

NLTK (Natural Language Toolkit) is one of the most popular and historically significant Python libraries for Natural Language Processing (NLP). It was built to provide a standard interface for working with human language data.

Core Features of NLTK

NLTK provides tools for almost every step of the traditional NLP pipeline:

- **Tokenization:** Breaking text into words or sentences.
- **Stopword Removal:** Removing common words (like "the", "is", "at") that add little meaning.
- **Stemming & Lemmatization:** Reducing words to their root form (e.g., "running" $\rightarrow$ "run").
- **Part-of-Speech (POS) Tagging:** Identifying if a word is a noun, verb, adjective, etc.
- **Named Entity Recognition (NER):** Detecting names, locations, and organizations.
- **Corpora Access:** Built-in access to datasets like the Gutenberg Bible, Brown Corpus, and Reuters.

2. Texts as Lists of Words

In NLTK and Python, a text (like a sentence, a paragraph, or an entire book) is simply stored as a List of individual words (tokens) and punctuation marks.

Example: *"Today is a nice day"*

['Today', 'is', 'a', 'nice', 'day']

This sentence has:

- 4 items (words + punctuation)
- Each item is a **string**

a) Creating and inspecting a list

Command	Simple English Meaning	Output/Example
sent1 = [‘Today’, ‘is’, ‘a’, ‘nice’, ‘day’, ‘.’]	Assigns the list to the variable sent1.	*No output, it saves the data.*
sent1	Shows you the contents of the list.	[‘Today’, ‘is’, ‘a’, ‘nice’, ‘day’,’.’]
len(sent1)	Counts how many items are in the list.	6

b) Combining and Adding to Lists

You can easily change the size of a list by joining them or adding a single item.

- **Concatenation (Joining Lists):** Uses the + sign.

  Python

  ```
  ['Hello'] + ['World', '!']
  # Output: ['Hello', 'World', '!']
  ```

- **Appending (Adding One Item):** Uses the `.append()` command.

  Python

  ```
  sent1 = ['Hello', 'World', '!']
  sent1.append('.')
  # sent1 is now: ['Hello', 'World', '!', '.']
  ```

- **Indexing and Slicing (Finding Specific Words)**

  Since a list is ordered, every word has an address, or index. The Golden Rule: Indexing Starts at Zero (0)

  Example: Sent1 = [‘Today’, ‘is’, ‘a’, ‘nice’, ‘day’,’.’]

Command	Simple English Meaning	Output/Example
Sent1[1]	Get one item at a specific position.	is
Sent1[0:3]	"Start at the beginning and go up to index 3."	‘Today’, ‘is’, ‘a’
Sent1[2:]	"Start at index 2 and go all the way to the end."	‘a’, ‘nice’, ‘day’,’.’

c) Strings

A string represents text as characters and list represented as words.

For Example:

String1 = "Python"

```
word[0]              # 'P'
word[1]              # 'y'
String1[:3]         # 'Pyt'
String1 + "!"       # 'Python!'
```

3. Simple Statistics

Statistical methods are fundamental to NLP as they provide quantitative insights into linguistic data. Even simple statistical analysis can reveal interesting patterns in text.

3.1 Frequency Distribution

A **frequency distribution** shows how often each vocabulary item appears in a text. This is one of the first steps in understanding any corpus (a collection of texts). It helps us identify which words are most common and which are rare.

For example, in English texts, words like *the*, *is*, and *and* often occur very frequently, whereas specific words like *machine*, *translation*, or *emotion* may appear less often.

Using NLTK in Python:

```
# Tokenizing
import nltk
from nltk.tokenize import word_tokenize
nltk.download('punkt_tab')
text = "It focuses on enabling computers to process, understand, and
generate human language. The phrase "Computing with Language" refers
to the computational treatment of linguistic data—how computers can
be programmed to handle text and speech in natural languages. In the
modern era, language data is abundant in the form of emails, web
pages, news articles, blogs, and social media posts. The challenge
is to convert this raw, unstructured text into structured data that
computers can understand and use for various applications such as"
tokens = word_tokenize(text)
print(tokens)


# Frequency Distribution Calculation

from nltk import FreqDist
fdist = FreqDist(tokens)
print(fdist.most_common(5))
```

This code will display the top five most frequent words in your text, such as:

```
[(',', 8), ('to', 4), ('and', 4), ('computers', 3), ('.', 3)]
```

By studying frequency distributions, we can identify dominant themes or key terms in a document. For instance, if we analyze a set of news articles, frequent words might tell us what topics are being discussed most often. It also helps in keyword extraction and topic modeling.

If we need to study **Hapaxes**, words or phrases that appear only *once* within a specific text:

```
print(fdist.hapaxes())
```

3.2 Fine-Grained Selection of Words

It focuses on long words of a text; perhaps these will be more characteristic and informative. We would like to find the words from the vocabulary of the text that are more than 7 characters long. Let's call this property P, so that P(w) is true if and only if w is more than 7 characters long. It can be representec using mathematical set notation:

a. $\{w \mid w \in V \ \& \ P(w)\}$

b. [w for w in V if p(w)]

This means "the set of all w such that w is an element of V (the vocabulary) and w has property P."

The corresponding Python expression is given:

```
V = set(text1)
long_words = [w for w in V if len(w) > 15]
sorted(long_words)
```

Result:

```
['Computing',
 'Language',
 'abundant',
 'applications',
 'articles',
 'challenge',
 'computational',
……................................]
```

3.3 Collocations and Bigrams

A collocation is a sequence of words that occur together unusually often. Thus, red wine is a collocation, whereas the wine is not. A characteristic of collocations is that they are resistant to substitution with words that have similar senses; for example, maroon wine sounds very odd. To get a handle on collocations, we start off by extracting from a text a list of word pairs, also known as bigrams. This is easily accomplished with the function bigrams ():

```
from nltk.collocations import BigramCollocationFinder
from nltk.metrics import BigramAssocMeasures

bigram_measures = BigramAssocMeasures()
finder = BigramCollocationFinder.from_words(tokens)

# Find the top 5 most frequent bigrams
print(finder.nbest(bigram_measures.pmi, 2))
```

Output

[('Computing', 'with'), ('It', 'focuses')]

Functions defined for NLTK's frequency distributions sre:

Example	Description
`fdist = FreqDist(samples)`	Create a frequency distribution containing the given samples
`fdist.inc(sample)`	Increment the count for this sample
`fdist['monstrous']`	Count of the number of times a given sample occurred
`fdist.freq('monstrous')`	Frequency of a given sample
`fdist.N()`	Total number of samples
`fdist.keys()`	The samples sorted in order of decreasing frequency
`for sample in fdist:`	Iterate over the samples, in order of decreasing frequency
`fdist.max()`	Sample with the greatest count
`fdist.tabulate()`	Tabulate the frequency distribution
`fdist.plot()`	Graphical plot of the frequency distribution
`fdist.plot(cumulative=True)`	Cumulative plot of the frequency distribution
`fdist1 < fdist2`	Test if samples in `fdist1` occur less frequently than in `fdist2`

4. Automatic Natural Language Understanding

Natural Language Understanding (NLU) is a crucial subfield of Natural Language Processing (NLP) that focuses on enabling computers to truly **understand** human language, not just process it. While NLP deals with analyzing the structure of language, NLU tries to interpret the **meaning** behind the words.

For example, humans can easily understand that the sentences *"I'm feeling blue"* and *"I'm sad"* convey the same meaning. For a machine, this requires more than just word matching — it needs to understand context, emotion, and intent.

NLU plays a key role in building intelligent systems like **chatbots, voice assistants, search engines, and question-answering systems**, where understanding user input is essential.

One of the long-standing goals of **Artificial Intelligence (AI)** is to build **intelligent machines**. Since language is central to human intelligence, understanding language is essential. Earlier,

this goal was considered too difficult. However, with modern NLP techniques and access to large amounts of data, **natural language understanding is now seen as achievable**, at least in restricted domains. A successful NLU should address many challenges.

a) Word Sense Disambiguation (WSD)

Word Sense Disambiguation (WSD) is a core challenge in Natural Language Processing (NLP). In simple terms, it is the ability of a computer program to identify which meaning of a word is being used in a sentence, based on the context.

The "Bank" Example

The classic example used to explain WSD is the word **"bank."**

- **Sentence A:** "I went to the **bank** to deposit my check."
- **Sentence B:** "We had a picnic and lot of fun at the **bank**."

In A, "bank" refers to a financial institution and in B, "bank" refers to sloping land beside a body of water.

This challenge can be solved by considering the context. Nearby words usually have related meanings. Humans use this naturally, and NLU systems must do the same. Understanding the meaning of surrounding words helps determine the correct sense.

b) Pronoun Resolution

Pronoun resolution in NLP is the process of determining the antecedent (the noun or noun phrase) that a pronoun (anaphor) refers to, which is a crucial task for understanding the meaning of a text.

Imagine you are trying to pack items.

Sentence A: "The trophy didn't fit into the brown suitcase because **it** was too **large**."

Sentence B: "The trophy didn't fit into the brown suitcase because **it** was too **small**."

The Resolution

- In **Sentence A**, "it" refers to the **Trophy**. (For something not to fit, the object going *inside* must be the large one).
- In **Sentence B**, "it" refers to the **Suitcase**. (If the container is too small, the object won't fit).

Grammatically, both sentences are identical. An algorithm cannot solve this by looking at noun positions or grammar rules alone. It must understand the **physics of the real world**:

- *Big objects don't fit in small containers.*

In this case, semantic role labeling identifying how a noun phrase relates to the verb.

c) Generating Language Output

Once the meaning is clear, we will be able to move on to tasks that involve generating language output, such as question answering and machine translation. In the first case, a machine should be able to answer a user's questions relating to collection of texts:

Example 1: Question Answering

Question: Where did the person go in Sentence A?
Generated Output: *To a bank (financial institution).*

Question: Where did the picnic take place in Sentence B?
Generated Output: *At the river bank.*

Example 2: Sentence Rewriting (Paraphrasing)

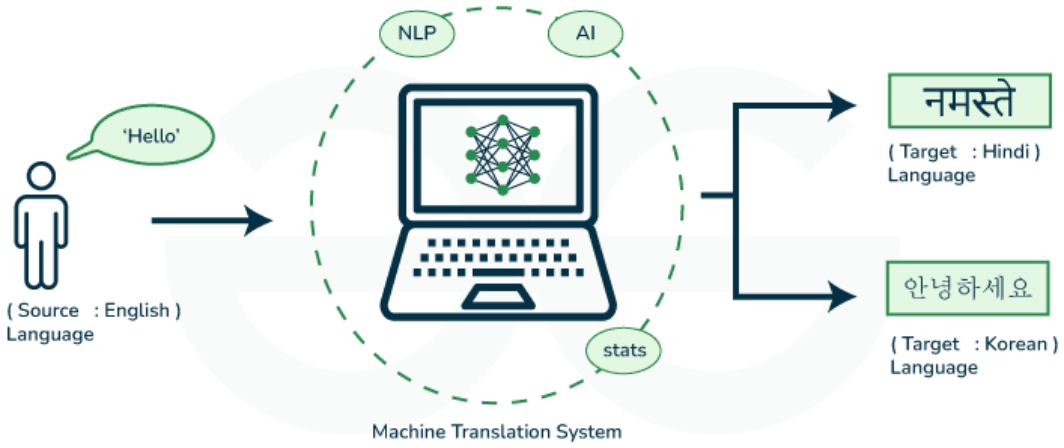
- **Sentence A (Generated):**
  I visited a financial bank to deposit money.

- **Sentence B (Generated):**
  We enjoyed a picnic near the river.

The response shows that the system was correctly able to catch the meaning based on the context. The paraphrasing required to rephrase the sentence by preserving its exact meaning. Without proper pronoun resolution, the translated sentence may convey an incorrect meaning.

d) Machine Translation

Machine translation of languages refers to the use of artificial intelligence (AI) and machine learning algorithms to automatically translate text or speech from one language to another.



Challenges in MT

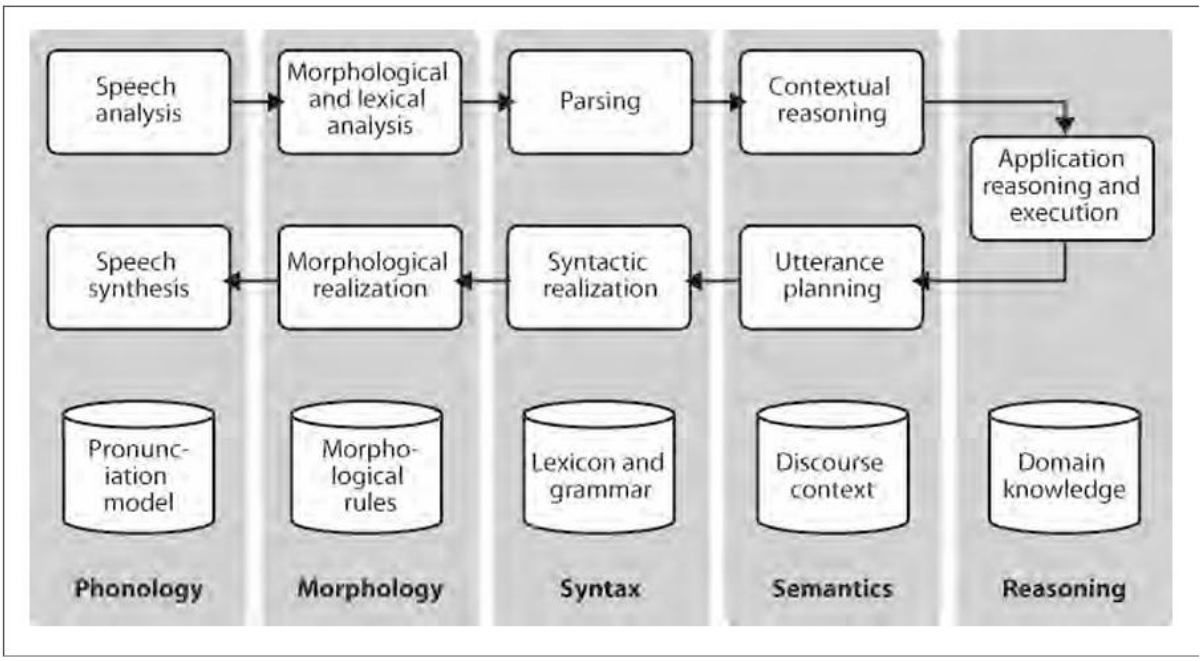
- Words have multiple meanings
- Word order (text alignment) differs across languages
  English - Subject-Verb-Object (SVO)
  Dravidian Languages - Subject-Object-Verb (SOV)
- Proper nouns may be misinterpreted
- Grammatical structure can be misunderstood

Text alignment is an important issue faced in machine translation. Text alignment of one language is mostly different from others. Translation need to preserve the meaning as well as the culture and alignment of the language.

e) Spoken Dialogue Systems

The historic measure of artificial intelligence is the **Turing Test**, Proposed by Alan Turing (1950). The test conducted to analyze can a computer dialogue system respond to text input so naturally that a human cannot distinguish its response from one generated by another human? Most of the systems focuses on specific domains. For example, a dialogue system build for agriculture wont work for healthcare.

Pipeline:



Understanding pipeline

- **Speech Analysis (Phonology) -** listens to **spoken input,** Converts into **phonetic units**
- **Morphological & Lexical Analysis (Morphology):** Breaks words into **meaningful units,** Identifies word forms (tense, plural, etc.)
- **Parsing (Syntax):** Analyzes sentence structure, Determines grammatical relationships
- **Contextual Reasoning (Semantics):** Understands meaning in context
- **Reasoning & Execution (Reasoning):** System decides what action to take

Generation Pipeline

- **Utterance Planning (Semantics → Syntax) –** Decides what to say

- **Syntactic Realization (Syntax) -** Converts meaning into grammatically correct sentences
- **Morphological Realization (Morphology) -** Applies correct word forms -Tense, agreement, pluralization
- **Speech Synthesis (Phonology) -** Converts text response into spoken output

f) Text entailment

Textual Entailment means checking whether one sentence (the hypothesis) is definitely supported by another sentence (the text).

Example 1:

Text:

Ravi won the first prize in the competition.

Hypothesis:

Ravi participated in the competition.

Entailment: Yes
If Ravi won, he must have participated.
The text entails the hypothesis.

Example 2:

Text:

Ravi participated in the competition.

Hypothesis:

Ravi won the first prize.

Entailment: No
Participation does not guarantee winning.
Not entailed

5. Accessing Text Corpora and Lexical Resources

In Natural Language Processing (NLP), a *text corpus* (plural: *corpora*) is a large and structured collection of texts used for analyzing, training, and evaluating language models. These texts may come from books, articles, websites, social media, transcripts, or domain-specific documents such as medical or legal records. Accessing text corpora refers to the methods and tools used to obtain, load, explore, and manage these collections so they can be effectively used in NLP tasks.

A corpus is not just a random collection of texts; it is usually organized, annotated, and representative of a language or domain.

- A *linguistic corpus* might be used to study grammar, syntax, or word usage.
- A *domain-specific corpus* might contain medical reports, legal documents, or tweets to study specialized language.

NLTK provides direct access to numerous text corpora and lexical resources that are essential for NLP experiments.

Corpora Available in NLTK

The **Natural Language Toolkit (NLTK)** provides several **pre-built text corpora** that can be directly accessed and analyzed. These corpora cover a variety of text types and genres.

a. **Gutenberg Corpus**
NLTK includes a small selection of texts from the Project Gutenberg electronic text archive, which contains some 25,000 free electronic books, hosted at http://www.gutenberg.org/

- To list all books available

```
import nltk
from nltk.corpus import gutenberg
nltk.download('gutenberg')
gutenberg.fileids()
```

- Words, Raw Text, and Sentences

```
gutenberg.raw('austen-emma.txt')
gutenberg.words('austen-emma.txt')
gutenberg.sents('austen-emma.txt')
```

- Counting words in a book

```
emma = gutenberg.words('austen-emma.txt')
len(emma)
```

- Understanding Corpus Statistics

```
num_chars = len(gutenberg.raw(fileid))
num_words = len(gutenberg.words(fileid))
num_sents = len(gutenberg.sents(fileid))
num_vocab = len(set(w.lower() for w in
gutenberg.words(fileid)))
```

Line	Meaning
num_chars	Total number of characters (letters + spaces)
num_words	Total number of words

Line	Meaning
num_sents	Total number of sentences
num_vocab	Number of **different** words

b. **Web Text Corpus (Informal Language)**

NLTK's also contains informal text contents such as Chats, Emails, Reviews, Forum posts. To access web text corpus:

```
from nltk.corpus import webtext
nltk.download('nps_chat')
webtext.fileids()
```

This corpus helps us study:

- Slang
- Short sentences
- Informal grammar

Important for:

- Chatbots
- Social media analysis

NPS chat: This corpus contains **real chat messages**.

```
from nltk.corpus import nps_chat
nltk.download('nps_chat')
nps_chat.posts()
```

c. **Brown Corpus**

One of the first electronic corpora of English, compiled in the 1960s. It contains 500 samples of English text from different categories like news, religion, fiction, and science.

To list categories:

```
from nltk.corpus import brown
nltk.download('brown')
brown.categories()
```

The Brown Corpus is a convenient resource for studying systematic differences between genres, a kind of linguistic inquiry known as stylistics. Let's compare genres in their usage of modal verbs.

d. **Reuters Corpus**

A collection of news documents, often used for **text classification** and **topic modeling**. Reuters corpus contains:

- News articles
- Each article has **topics**

To list news are article

```
from nltk.corpus import reuters
nltk.download('reuters')
reuters.categories()
```

e. **Inaugural Address Corpus**
Contains all U.S. presidential inaugural speeches from George Washington to recent presidents. Useful for analyzing changes in political language over time.
To list the speeches:

```
from nltk.corpus import inaugural
nltk.download('inaugural')
inaugural.fileids()
```

Common Corpus Methods

NLTK provides a few simple but powerful methods to access data from any corpus:

Method	Description	Example Usage
`.words()`	Returns a list of all words (tokens) in the corpus	`gutenberg.words('austen-emma.txt')`
`.sents()`	Returns a list of sentences (each sentence is a list of words)	`gutenberg.sents('austen-emma.txt')`
`.raw()`	Returns the entire text as a single string (useful for raw text processing)	`gutenberg.raw('austen-emma.txt')`

Annotated Text Corpus

Annotated corpora contain linguistic metadata beyond just raw text and word tokens. This metadata is crucial for training and evaluating NLP models.

Annotation Type	Description
Part-of-Speech (POS) Tags	Labels words with their grammatical role (e.g., Noun, Verb, Adjective).
Named Entities	Identifies and classifies proper nouns (e.g., Person, Location, Organization).
Syntactic Structures	Provides the grammatical structure of sentences, often as a tree representation.
Semantic Roles/Propositions	Labels the "who did what to whom" in a sentence.
Sentiment Polarity	Classifies text as positive or negative (or neutral).

Corpora in Other Languages

Corpus	Language(s)	Key Feature	Access Example
CESS Treebanks	Spanish, Catalan	Tagged and parsed.	nltk.corpus.cess_esp.words()
Floresta Treebank	Portuguese	Tagged and parsed.	nltk.corpus.floresta.words()
Indian POS Tagged Corpus	Bangla, Hindi, Marathi, Telugu	Contains POS-tagged text in several Indian languages.	nltk.corpus.indian.words('hindi.pos')
Universal Declaration of Human Rights (UDHR)	Over 300 languages	Text is available in many languages and encodings.	udhr.fileids()

6. Conditional Frequency Distributions

A **Conditional Frequency Distribution (CFD)** is a statistical tool used to analyze the relationship between two variables. Essentially, it is a collection of frequency distributions for one variable, where each individual frequency distribution is calculated under a specific *condition* imposed by the other variable.

CFD = *"How often does X occur **when** Y is true?"*

How it works:

The primary use of a CFD is to understand how the distribution of one variable change when you limit your focus to a specific subgroup defined by another variable. This allows you to identify patterns, associations, or dependencies between the two variables.

Following is the example for conditional random field. Here the subject preference varies or dependent on the gender.

Gender	Cricket	Badminton	Football
Male	10	8	20
Female	8	12	5

$$\mathrm{P(Basketball|Female)} = \frac{\text{Count of Females who prefer Basketball}}{\text{Total number of Females surveyed}}$$

For example:

```
cfd = nltk.ConditionalFreqDist((genre, word.lower())
    for genre in brown.categories()
```

```
    for word in brown.words(categories=genre)
)
# Frequency of the word "the" in different genres
print("Frequency of the word 'the' in news Category - ",cfd['news']['the'])
print("Frequency of the word 'the' in news Category-", cfd['romance']['the'])
```

Output:

Frequency of the word 'the' in news Category - 6386
Frequency of the word 'the' in news Category- 2988

CFDs are powerful for analyzing relationships between linguistic variables — for example, comparing how often certain words or grammatical forms appear across genres or contexts.

7. Lexical Resources

A **lexicon**, or **lexical resource**, is a systematic collection of words and/or phrases along with associated linguistic information. Unlike a text corpus (which is a raw collection of text), a lexicon is a *secondary* resource that organizes and annotates language data.

This associated information can include:

- **Part-of-speech (POS)**: Identifying if a word is a noun, verb, adjective, etc.
- **Sense definitions (gloss)**: The meaning(s) of the word.

Raw collection of text is run and happy its lexical resources are given below:

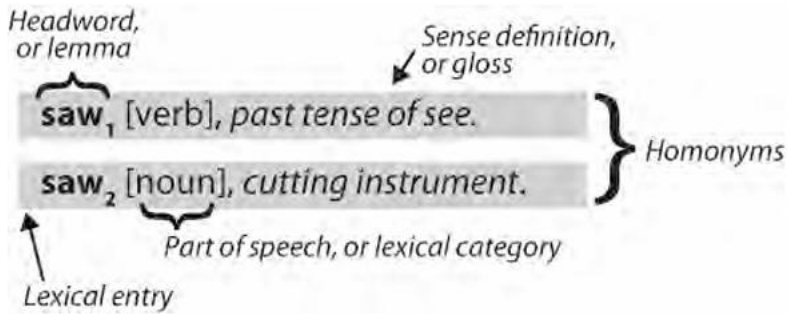
Example 1:

Word	POS	Meaning
run	Verb	to move fast on foot

Example 2:

Word	Synonyms	Antonyms
happy	joyful, glad, pleased	sad, unhappy

The text describes several types of lexical resources available in NLTK, ranging from simple wordlists to complex pronouncing dictionaries and comparative wordlists. Homonyms are different words that share the same spelling. They have different meanings and possibly different POS. The meaning in particular context can be identified by the associated POS tag.



Different lexical resources available in NLTK:

- **Wordlists:** NLTK includes some corpora that are nothing more than wordlists.

- **Stopwords:** stopwords are the high-frequency words such as *the, to, across, also* that we sometimes want to filter out of a document before further processing. Stopwords usually have little lexical content, and their presence in a text fails to distinguish it from other texts.

```
from nltk.corpus
import stopwords
stopwords.words('english')
```

Output:

```
['a',    "a's",    'able',    'about',    'above',    'according',
'accordingly',   'across','actually',   'after',   'afterwards',
'again', 'against', "ain't", 'all', 'allow', 'allows', 'almost',
'alone', 'along', 'already', 'also', 'although', 'always', ...]
```

- **Pronouncing Dictionary**

A Pronouncing Dictionary is a specialized type of lexical resource that focuses almost exclusively on providing the correct or accepted ways to articulate words. Unlike a general-purpose dictionary, the primary purpose of a pronouncing dictionary is to serve as an authoritative guide to pronunciation. **Phone** is smallest identifiable unit in the flow of speech. This database provides distinct labels for each phone.

For example: Lexical
phonetic dictionary: **lek**·suh·kl

- **Comparative Wordlists**

Another example of a tabular lexicon is the comparative wordlist. NLTK includes so-called Swadesh wordlists, lists of about 200 common words in several languages. The languages are identified using an ISO 639 two-letter code.

```
import nltk
from nltk.corpus import swadesh
nltk.download('swadesh')
swadesh.fileids()
```

Output:

['be', 'bg', 'bs', 'ca', 'cs', 'cu', 'de', 'en', 'es', 'fr', 'hr', 'it', 'la', 'mk', 'nl', 'pl', 'pt', 'ro', 'ru', 'sk', 'sl', 'sr', 'sw', 'uk']

- **Shoebox and Toolbox** are classic, powerful, free linguistics software from SIL International (SIL) for field linguists to create, manage, and analyze lexicons (dictionaries)

WordNet

WordNet is one of the most important lexical databases in NLP. It groups words into sets of synonyms called **synsets** and defines relations among them such as:

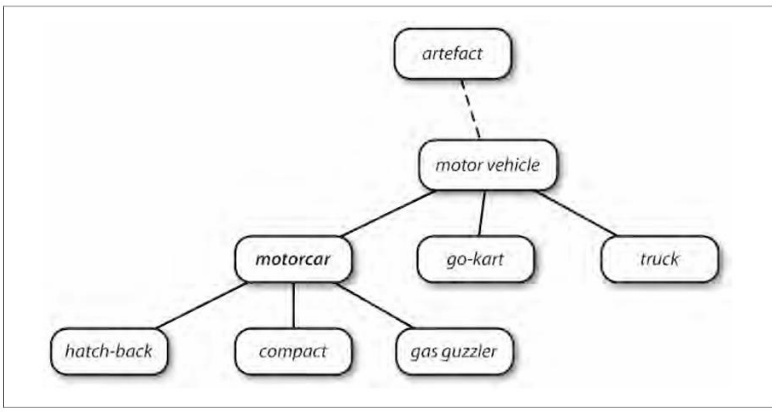
- **Synonymy** – words with similar meanings
- **Antonymy** – words with opposite meanings
- **Hyponymy/Hypernymy** – "is-a" relationships (e.g., dog is a type of animal)
- **Meronymy/Holonymy** – part-whole relationships (e.g., wheel is part of car)

WordNet Applications:

- Word Sense Disambiguation
- Semantic Similarity Measurement
- Information Retrieval and Summarization

The WordNet Hierarchy

Wordnet concept can be linked together in hierarchy. Some concepts are very general, such as Entity, State, Event; these are called **unique beginners** or **root synsets**. The following graph shows **hypernyms and hyponyms. Hypernyms** are broader, more general terms (e.g., motor vehicle), while **hyponyms** are more specific terms (e.g., "motorcar"), forming hierarchical "is-a" relationships.



Important points in wordnet heirarchy:
Hypernymy and Hyponymy
This relationship describes a hierarchy of categories, a "kind-of" relationship.

- **Hypernym (Superordinate term):** A general or generic term (an "umbrella term") that names a broad category.
- **Hyponym (Subordinate term):** A more specific term that falls within the category of a hypernym. The hyponym "is a kind of" the hypernym.

- **Example:**
    - **Animal** is a hypernym of **dog**.
    - **Dog** is a hyponym of **animal**.
    - **Fruit** is a hypernym of **apple**, **orange**, and **banana**.

Meronymy and Holonymy

This relationship describes a compositional structure, a "part-of" or "member-of" relationship.

- **Holonym:** The term for the whole object or concept.
- **Meronym:** A term for a constituent part or member of the whole. The meronym "is a part of" the holonym.
- **Example:**
    - **Car** is a holonym of **wheel**, **engine**, and **door**.
    - **Wheel** is a meronym of **car**.
    - **Tree** is a holonym of **branch**, **leaf**, and **root**.

****

Question Bank

Part A

1. Natural Language Processing (NLP) is a field that combines

a) Computer science and mathematics
b) Linguistics and psychology
c) Computer science, linguistics, and machine learning
d) Artificial intelligence and robotics

2. The phrase *"Computing with Language"* refers to

a) Programming languages only
b) Computational treatment of linguistic data
c) Machine-level instructions
d) Speech signal processing

3. Which of the following is NOT an application of NLP?

a) Chatbots
b) Sentiment analysis
c) Image compression
d) Machine translation

4. Which programming language is widely preferred for NLP?

a) C
b) Java

c) Python
d) Assembly

5. NLTK stands for

a) Natural Language Tool Kit
b) Natural Linguistic Toolkit
c) Neural Language Toolkit
d) Natural Language Toolkit

6. Tokenization refers to

a) Removing stopwords
b) Splitting text into tokens
c) Assigning POS tags
d) Extracting entities

7. Which of the following is an example of a token list?

a) "Today is a nice day"
b) {Today, is, a, nice, day}
c) ['Today', 'is', 'a', 'nice', 'day']
d) (Today, is, a, nice, day)

8. In Python, list indexing starts from

a) 1
b) 0
c) –1
d) Any number

9. The command len(sent1) is used to

a) Display elements
b) Append items
c) Count number of elements
d) Sort the list

10. Which method adds a single item to a list?

a) extend()
b) add()
c) insert()
d) append()

11. A frequency distribution shows

a) Sentence structure
b) Word meanings

c) Word occurrence counts
d) Grammar rules

12. Which NLTK class is used to compute frequency distribution?

a) Counter
b) WordDist
c) FreqDist
d) TextDist

13. Hapaxes are words that

a) Occur very frequently
b) Occur only once
c) Occur twice
d) Are stopwords

14. Collocations are

a) Random word pairs
b) Frequently co-occurring word sequences
c) Synonyms
d) Antonyms

15. The function used to extract bigrams is

a) trigrams()
b) ngrams()
c) bigrams()
d) collocations()

16. Natural Language Understanding (NLU) mainly focuses on

a) Text formatting
b) Meaning and interpretation
c) Speech synthesis
d) Token generation

17. Word Sense Disambiguation deals with

a) Sentence structure
b) Multiple meanings of a word
c) Word pronunciation
d) Stopword removal

18. Pronoun resolution identifies

a) Verb tense
b) Word category

c) Antecedent of a pronoun
d) Sentence type

19. The Turing Test was proposed by

a) John McCarthy
b) Alan Turing
c) Noam Chomsky
d) Marvin Minsky

20. Machine Translation mainly converts

a) Images to text
b) Speech to text
c) One language to another
d) Text to speech

21. A text corpus is

a) A single document
b) A programming file
c) A structured collection of texts
d) A dictionary

22. Which corpus contains informal language like chats and forums?

a) Brown corpus
b) Gutenberg corpus
c) Web text corpus
d) Reuters corpus

23. Conditional Frequency Distribution (CFD) is used to analyze

a) Single variable statistics
b) Relationship between two variables
c) Sentence length
d) Word pronunciation

24. WordNet groups words into

a) Tokens
b) Wordlists
c) Synsets
d) Sentences

25. Hypernym–Hyponym relationship represents

a) Part–whole relation
b) Synonym relation

c) Opposite meaning
d) “Is-a” hierarchy

26. What is the primary focus of semantic analysis?

a. Grammar rules
b. Sentence length
c. Word meanings & relationships
d. Text formatting

Answer Key – Part A

1. c
2. b
3. c
4. c
5. d
6. b
7. c
8. b
9. c
10. d
11. c
12. c
13. b
14. b
15. c
16. b
17. b

18. c
19. b
20. c
21. c
22. c
23. b
24. c
25. d
26. c

PART B – 5 Mark Questions

1. Define Natural Language Processing and list its major applications.
2. Explain the role of NLTK in Natural Language Processing.
3. Describe how text is represented as a list of words in Python with an example.
4. Explain frequency distribution and its importance in NLP.
5. What are collocations? Explain with examples.
6. Describe Word Sense Disambiguation with a suitable example.
7. Explain pronoun resolution using suitable sentences.
8. What is a text corpus? Explain its importance in NLP.
9. Write a short note on Conditional Frequency Distribution (CFD).
10. Explain stopwords and their role in text processing.

PART C – 8 Mark Questions

1. Explain the NLP pipeline and its major components in detail.
2. Discuss Natural Language Understanding (NLU) and its challenges.

3. Describe frequency distribution, hapaxes, and fine-grained word selection with examples.

4. Explain collocations and bigram extraction using NLTK.

5. Discuss Machine Translation and its challenges.

6. Explain different types of corpora available in NLTK.

7. Describe Conditional Frequency Distribution with a suitable example.

8. Explain lexical resources and their importance in NLP.

9. Discuss WordNet, its structure, and applications.

10. Explain the Turing Test and its significance in spoken dialogue systems.